

CURSOR IMPLEMENTATION PROMPTS

Trustworthy AI for Protein Engineering

Discovering Mutation-Space Reliability and Applying It to Industrial Carbon-Capture Enzyme Design

Version 2.0

Purpose Of This Document

This document contains the exact sequence of implementation prompts to be provided to Cursor.

Each prompt represents a complete development phase.

Do not skip phases.

Do not combine phases.

Each phase must be:

Implemented

Tested

Validated

Committed

Documented

``

before moving to the next prompt.

The project follows:

Infrastructure

↓

Datasets



Sparsity



Experiment 1



Experiment 2



Reliability



TKT Engineering



Industrial Fitness



Publication Assets

PROMPT 1

Infrastructure Foundation

Objective

Build the complete project foundation.

Cursor Prompt

Create the complete infrastructure foundation for the Trustworthy AI for Protein Engineering project.

Requirements:

1. Create directory structure:

configs/
data/
docs/
notebooks/
outputs/
reports/
src/
tests/

2. Create project-wide configuration framework.
3. Create logging framework with console and file logging.
4. Create experiment tracking module.
5. Create dataset registry.
6. Create validation utilities.
7. Create reusable utility package.
8. Create unit testing framework.
9. Create project bootstrap script.
10. Create documentation for all modules.

Requirements:

- Python 3.11+
- Type hints everywhere
- Pydantic validation
- Modular architecture
- Comprehensive docstrings
- Production-quality code

Deliverables:

Project boots successfully.

Provide complete code for every file.

Completion Test

Project starts successfully.

Logging works.

Tests execute.

Configuration loads.

PROMPT 2

Dataset Framework

Objective

Build dataset ingestion and validation systems.

Cursor Prompt

Implement the Dataset Framework.

Requirements:

1. Dataset loading system.
2. Schema registry system.
3. Dataset validation framework.
4. Version tracking.
5. Read/write abstraction layer.
6. Dataset metadata management.

Implement support for:

megascale_d.parquet
protein_sequences.parquet
protein_structures.parquet

Validation rules:

- Missing values
- Duplicate keys
- Data type validation
- Range validation
- Schema validation

Generate complete source code.

Generate automated tests.

Generate documentation.

Provide all files.

Completion Test

All source datasets load.

Validation passes.

Metadata recorded.

PROMPT 3

Empirical Sparsity Framework

Objective

Implement empirical sparsity calculations.

Cursor Prompt

Implement the Empirical Sparsity module.

Inputs:

Megascale-D mutation dataset.

Requirements:

1. Calculate mutation frequency.
2. Calculate normalized frequency.
3. Compute empirical sparsity.

Formula:

Empirical Sparsity =
1 - Normalized Frequency

Outputs:

empirical_sparsity.parquet

Required fields:

mutation_id
frequency
normalized_frequency
empirical_sparsity

Generate:

Production code
Validation tests
Documentation
Example notebook

Provide all files.

Completion Test

empirical_sparsity.parquet
generated successfully.

PROMPT 4

Evolutionary Sparsity Framework

Objective

Implement ESM2-based evolutionary sparsity.

Cursor Prompt

Implement Evolutionary Sparsity.

Requirements:

1. Load ESM2 model.
2. Score all mutations.
3. Calculate mutation probabilities.
4. Convert probabilities into sparsity.

Formula:

Evolutionary Sparsity =
Normalized(-log(P))

Output:

evolutionary_sparsity.parquet

Fields:

mutation_id
esm_probability
log_probability
evolutionary_sparsity

Generate:

Source code
Tests
Documentation
Example usage

Provide all files.

,

Completion Test

Evolutionary sparsity values
successfully generated.

PROMPT 5

Structural Sparsity Framework

Objective

Implement SASA-based structural sparsity.

Cursor Prompt

Implement Structural Sparsity.

Requirements:

1. Load protein structures.
2. Compute SASA values.
3. Normalize SASA measurements.
4. Calculate structural sparsity.

Formula:

Structural Sparsity =
 $1 - \text{Normalized SASA}$

Output:

structural_sparsity.parquet

Fields:

mutation_id
sasa
normalized_sasa
structural_sparsity

Generate:

Production code
Unit tests
Documentation
Validation reports

Provide all files.

Completion Test

structural_sparsity.parquet
created successfully.

PROMPT 6

Combined Sparsity Framework

Objective

Generate the unified sparsity metric.

Cursor Prompt

Build the Combined Sparsity Framework.

Inputs:

Empirical Sparsity
Evolutionary Sparsity
Structural Sparsity

Implement:

Megascale-D mode:

Combined =
 $(E + \text{Evo} + \text{Struct}) / 3$

TKT mode:

Combined =
(Evo + Struct) / 2

Generate:

combined_sparsity.parquet

Fields:

mutation_id
empirical_sparsity
evolutionary_sparsity
structural_sparsity
combined_sparsity

Add all validation tests.

Provide all code.

Completion Test

combined_sparsity.parquet
created.

PROMPT 7

Experiment 1 – Antisymmetry Analysis

Objective

Implement Experiment 1.

Cursor Prompt

Implement Experiment 1:
Sparsity and Thermodynamic Inconsistency.

Requirements:

1. Forward mutation prediction pipeline.
2. Reverse mutation prediction pipeline.
3. Antisymmetry calculations.

Formula:

Antisymmetry Error =
| Forward + Reverse |

Outputs:

antisymmetry_predictions.parquet

antisymmetry_results.parquet

Required analyses:

- Correlation
- Regression
- Quantile comparison
- Distribution plots

Generate publication-quality figures.

Provide all code.

Completion Test

Antisymmetry analyses complete.

Figures generated.

PROMPT 8

Experiment 2 – Epistasis Analysis

Objective

Implement validation experiment.

Cursor Prompt

Implement Experiment 2:
Sparsity and Epistasis Prediction Error.

Requirements:

1. Experimental epistasis calculation.
2. Predicted epistasis calculation.
3. Epistasis error calculation.

Formula:

Error =
 $| \text{Predicted} - \text{Experimental} |$

Output:

epistasis_results.parquet

Generate:

- Statistical analysis
- Correlations
- Regression models
- Validation figures

Provide complete implementation.

Completion Test

epistasis_results.parquet
generated.

PROMPT 9

Integrated Reliability Analysis

Objective

Build the scientific centerpiece.

Cursor Prompt

Implement Integrated Reliability Analysis.

Evaluate:

Combined Sparsity

↓

Antisymmetry Error

↓

Epistasis Error

Requirements:

1. Integrated dataset generation.
2. Relationship analysis.
3. Multi-stage regression.
4. Visualization generation.

Output:

integrated_reliability_analysis.parquet

Generate publication figures.

Provide all files.

Completion Test

Integrated analysis complete.

PROMPT 10

Reliability Framework

Objective

Create reliability estimation system.

Cursor Prompt

Implement Reliability Framework.

Formula:

Reliability =
 $1 - \text{Combined Sparsity}$

Generate:

reliability_scores.parquet

Fields:

mutation_id
reliability_score
reliability_category

Categories:

High
Moderate
Low
Very Low

Generate:

Visualizations
Validation tests
Documentation

Provide all code.

Completion Test

Reliability framework operational.

PROMPT 11

Complete TKT Mutation Landscape

Objective

Generate every possible TKT mutation.

Cursor Prompt

Implement Complete TKT Mutation Landscape generation.

Requirements:

1. Load TKT sequence.
2. Enumerate all possible single mutations.
3. Generate mutation catalogue.
4. Calculate:

- Evolutionary Sparsity
- Structural Sparsity
- Combined Sparsity
- Reliability Score

Outputs:

tkt_single_mutation_landscape.parquet

tkt_mutation_analysis.parquet

Generate complete implementation.

Provide all files.

Completion Test

Complete mutation landscape generated.

PROMPT 12

Active-Site Control Analysis

Objective

Implement control analysis.

Cursor Prompt

Implement Active-Site Control Analysis.

Requirements:

1. Define active-site residues.
2. Calculate residue distances.
3. Calculate minimum active-site distance.
4. Evaluate relationship between:

Distance

Combined Sparsity

Reliability

Output:

active_site_analysis.parquet

Generate:

Figures

Statistical analyses

Documentation

Provide all code.

Completion Test

Active-site analyses complete.

PROMPT 13

Industrial Fitness Framework

Objective

Implement engineering ranking system.

Cursor Prompt

Implement Industrial Fitness Framework.

Formula:

$$\begin{aligned} \text{Fitness} = & \\ & 0.50 \times \text{Stability} \\ & + \\ & 0.35 \times \text{Reliability} \\ & + \\ & 0.15 \times \text{Evolutionary Plausibility} \end{aligned}$$

Requirements:

1. Score every mutation.
2. Rank mutations.
3. Generate candidate tables.
4. Generate engineering reports.

Outputs:

industrial_fitness_scores.parquet

top_candidate_mutations.csv

Provide all implementation files.

Completion Test

Candidate rankings generated.

PROMPT 14

Publication Figures

Objective

Generate all project figures.

Cursor Prompt

Generate publication-ready figures.

Required figure groups:

1. Discovery Figures
2. Validation Figures
3. Reliability Figures
4. TKT Landscape Figures
5. Industrial Fitness Figures

Requirements:

- Publication quality
- Consistent styling
- Export PNG and PDF
- Figure captions included

Provide all code.

Completion Test

All figures generated.

PROMPT 15

Killer Figure

Objective

Generate the flagship project figure.

Cursor Prompt

Create the flagship figure for the project.

The figure must communicate:

Megascale-D

↓

Combined Sparsity

↓

Antisymmetry Error

↓

Epistasis Error

↓

Reliability Framework

↓

Complete TKT Mutation Landscape

↓

Industrial Fitness Ranking

↓

Top Candidate Mutations

Requirements:

- Science fair ready

- Publication ready
- Presentation ready
- High visual quality
- Minimal text
- Easy to understand in 10 seconds

Generate complete figure-generation code.

Provide all files.

Completion Test

Killer Figure generated successfully.

Final Cursor Completion Checklist

- ✓ Infrastructure Complete
- ✓ Dataset Framework Complete
- ✓ Sparsity Framework Complete
- ✓ Experiment 1 Complete
- ✓ Experiment 2 Complete
- ✓ Integrated Reliability Analysis Complete
- ✓ Reliability Framework Complete
- ✓ Complete TKT Mutation Landscape Complete
- ✓ Active-Site Analysis Complete
- ✓ Industrial Fitness Framework Complete
- ✓ Candidate Rankings Generated
- ✓ Publication Figures Generated

End Of Cursor Implementation Prompts Guide

The next and final document is:

SYSTEM OPERATION,
TESTING & VALIDATION HANDBOOK

This will define exactly:

What To Run

What Output To Expect

How To Verify Results

Pass/Fail Criteria

Troubleshooting Steps

Validation Procedures

for every phase of the project.